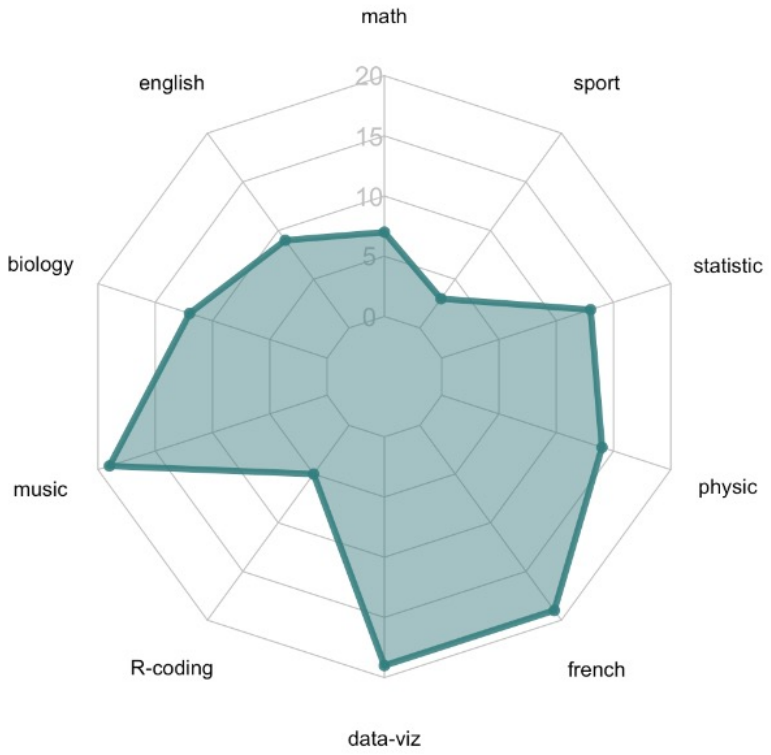
Radar / Spider / Web chart

Giảng viên: Nguyễn Ngọc Giang

giang.nguyenngoc@phenikaa-uni.edu.vn

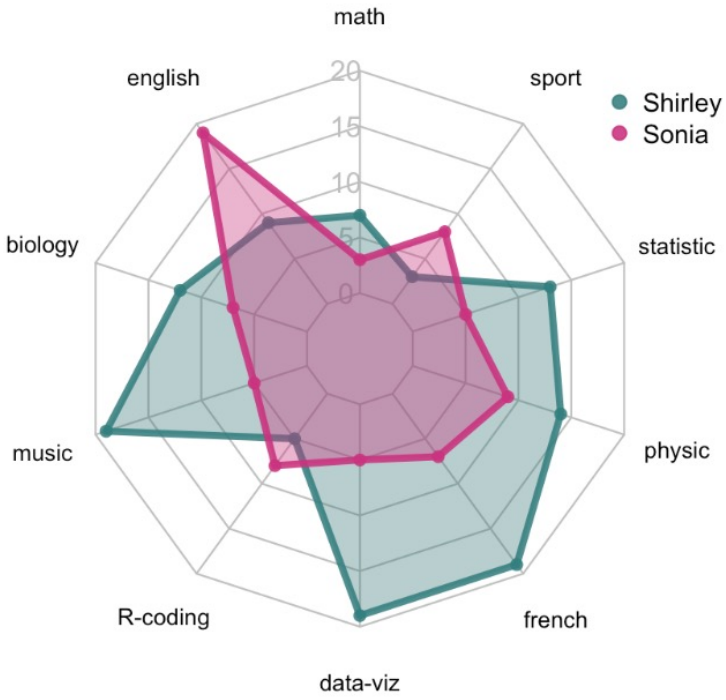
1. Khái niệm

- Biểu đồ radar hoặc spider hoặc web là loại biểu đồ 2-D được thiết kế để vẽ một hoặc nhiều chuỗi giá trị trên nhiều biến định lượng. Mỗi biến có trục riêng, tất cả các trục được nối ở giữa hình.
- Vd: Xét điểm thi của 1 học sinh trong các môn toán, thể thao, thống kê,… Và có khoảng điểm từ 0 đến 20.
- Biểu đồ radar cung cấp một trục cho mỗi chủ đề. Hình dạng này cho phép bạn xem học sinh thực hiện tốt hay kém chủ đề nào.



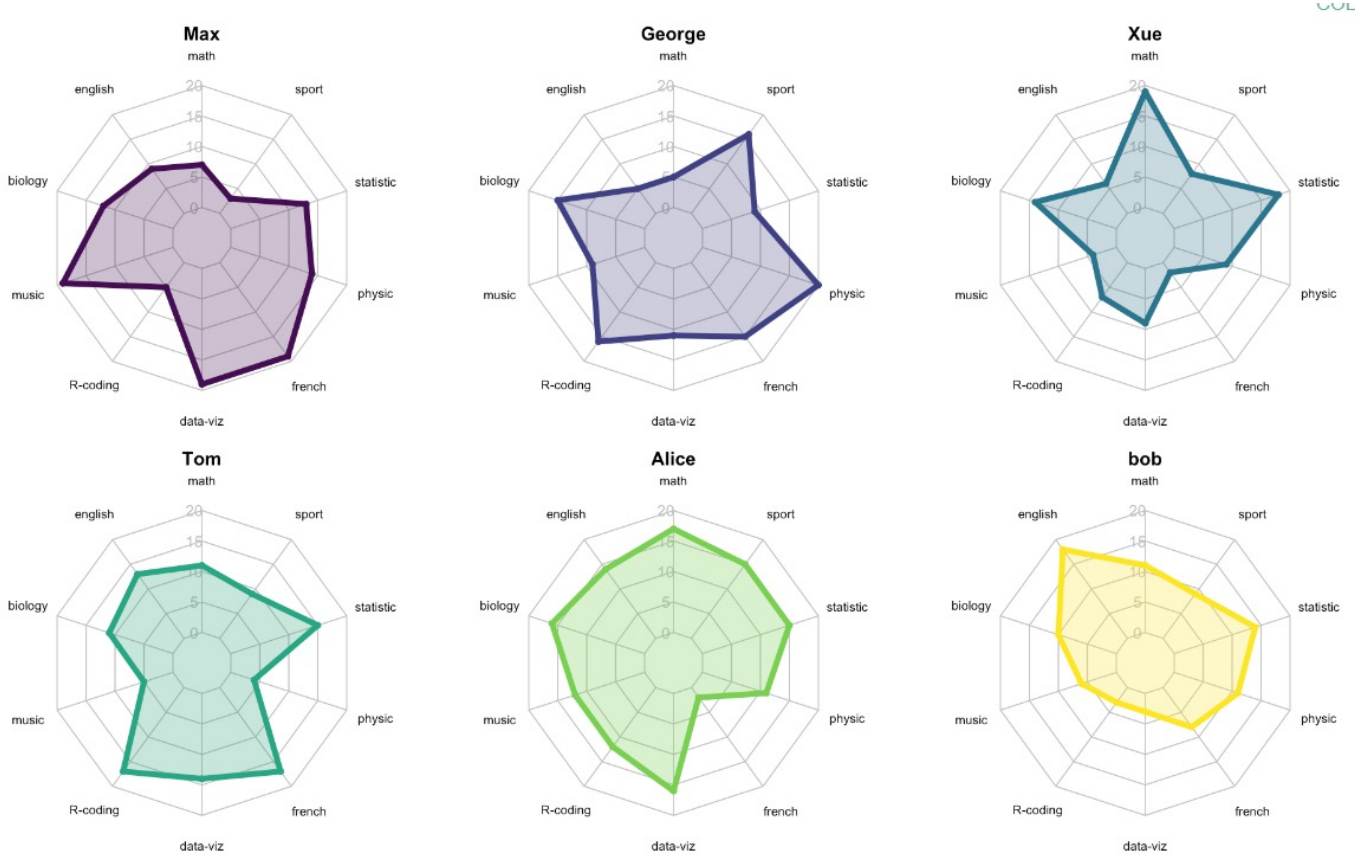
1. Khái niệm

- VD này so sánh 2 học sinh trên cùng 1 biểu đồ.
- Ta thấy rõ rằng Shirley trên toàn bộ các môn hoạt động tốt hơn Sonia, ngoại trừ môn thể thao, tiếng Anh và mã R.



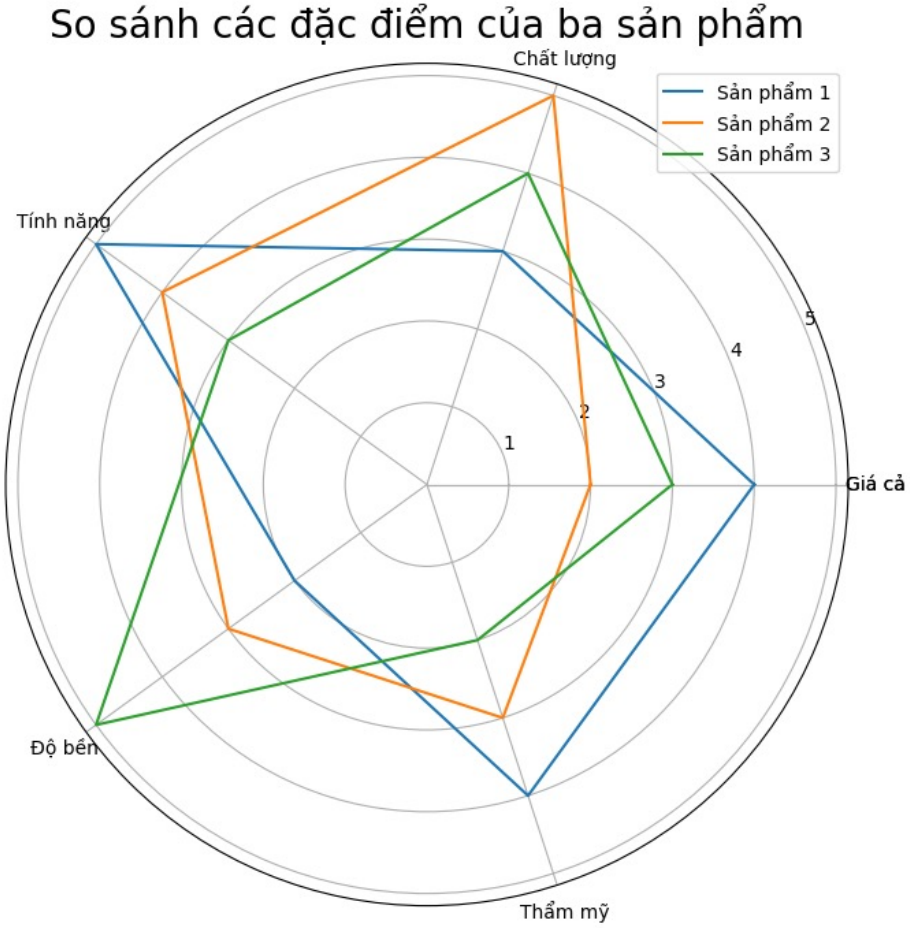
1. Khái niệm

Với nhiều hơn hai hoặc ba chuỗi, ta nên vẽ riêng biệt để có thấy rõ đặc điểm của từng biến. Và cũng sẽ dễ dàng tìm được sự tương đồng về hình dạng.



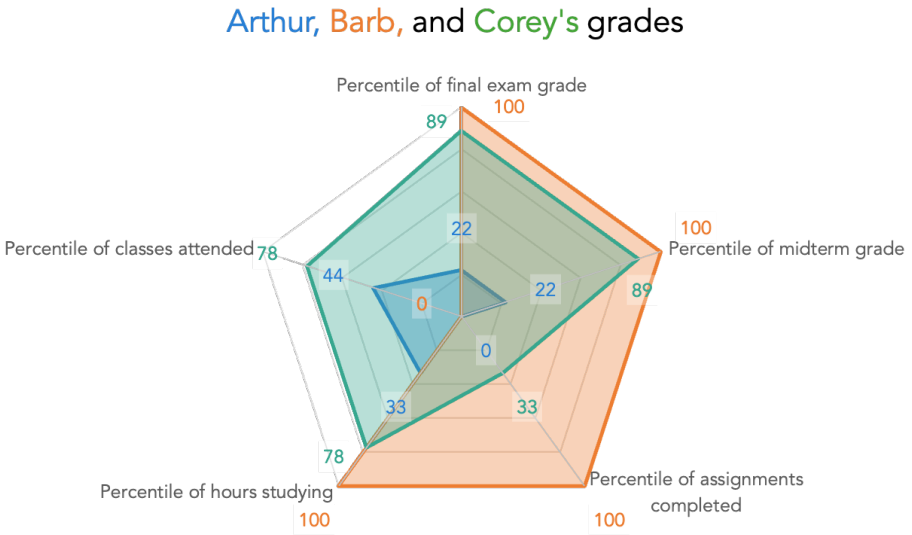
2. Ứng dụng

- So sánh hiệu suất giữa các thực thể.
- Thể hiện sự tương quan giữa các biến đa chiều.
- Thường sử dụng trong phân tích hiệu suất, đánh giá kỹ năng, sản phẩm.



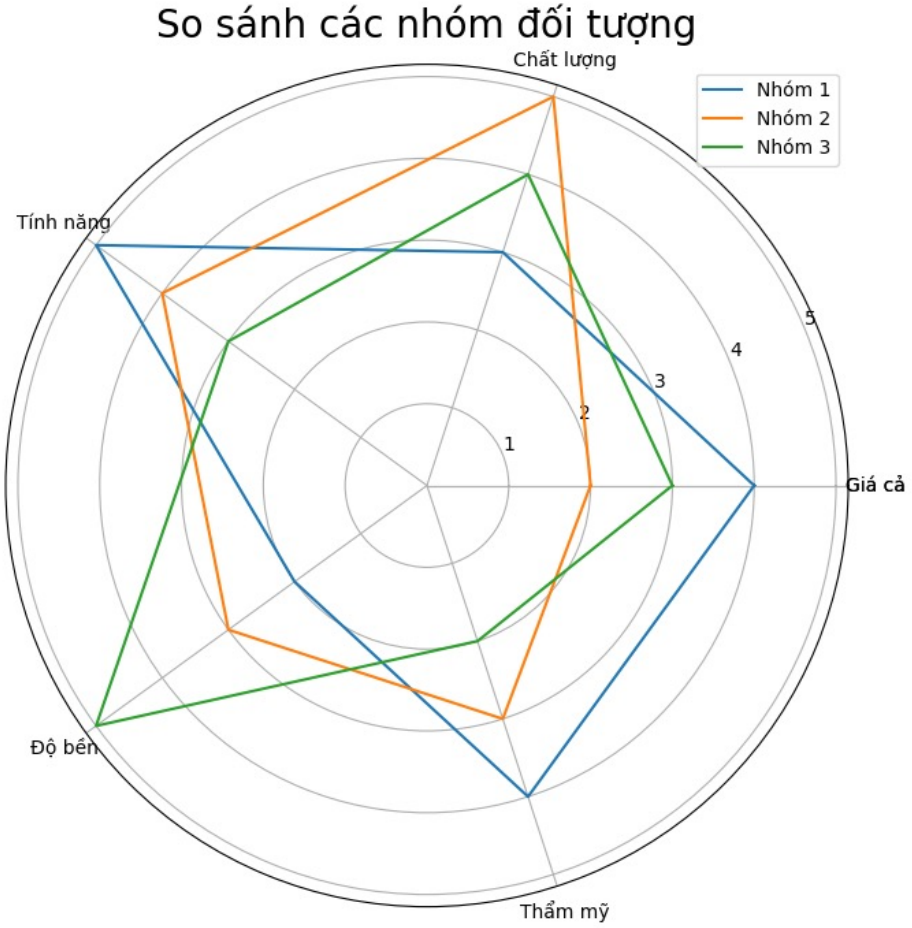
3. Cấu trúc của Spider chart

- **Trục (Axes):** Mỗi trục đại diện cho một biến trong tập dữ liệu. Trục được sắp xếp đều quanh một điểm trung tâm.
- **Điểm dữ liệu (Data Points):** Giá trị của mỗi biến được đánh dấu trên trục tương ứng.
- **Đường kết nối (Connecting Lines):** Các điểm dữ liệu trên các trục được kết nối với nhau để tạo ra một hình dạng đóng (polygon).
- **Vùng màu (Shading Area):** Một số biểu đồ Spider sử dụng màu sắc để làm nổi bật khu vực được bao phủ bởi các điểm dữ liệu, giúp dễ dàng nhận diện sự khác biệt giữa các thực thể.



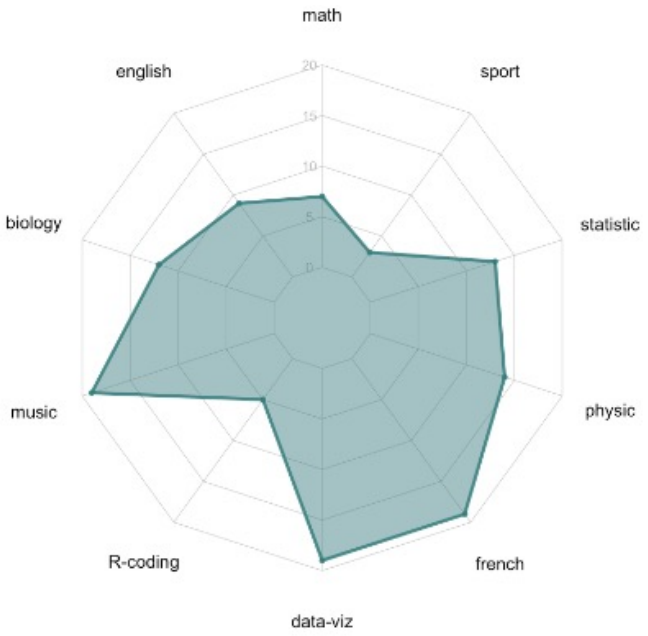
4. Ưu điểm của Spider chart

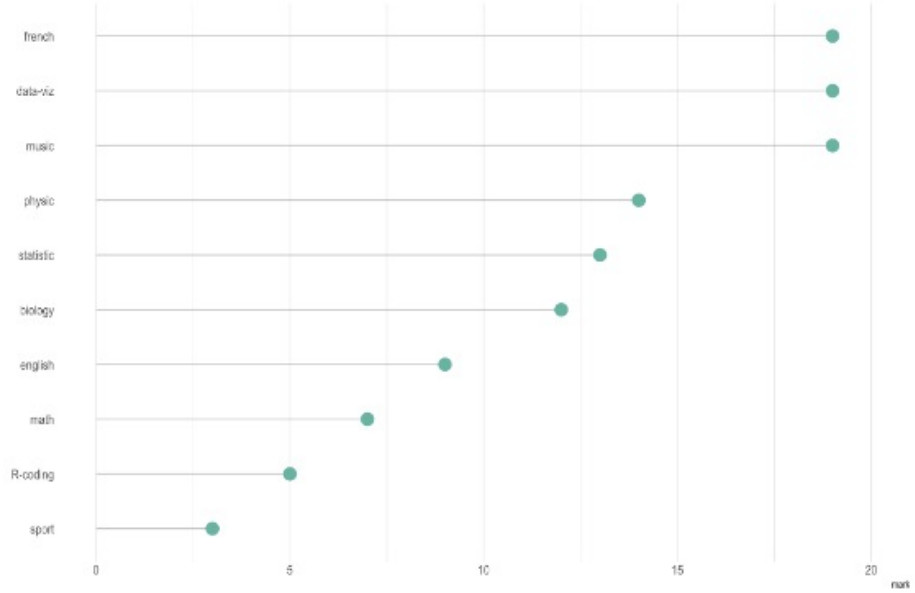
- Dễ nhìn thấy sự khác biệt giữa các thực thể qua các biến số.
- Hữu ích để so sánh nhiều biến trên một biểu đồ duy nhất.
- Trực quan và dễ hiểu cho các nhóm đối tượng khác nhau.



5. Nhược điểm của Spider chart

1) Bố cục hình tròn = khó đọc hơn - các giá trị định lượng dễ so sánh hơn nhiều khi chúng được bố trí dọc theo một trục dọc hoặc trục ngang. Đây là một lời chê trách chung đối với bố cục hình tròn. So sánh hình dưới đây chỉ xem xét dữ liệu của một học sinh. Việc so sánh các giá trị trong lollipop chart, barplot sẽ dễ dàng hơn và chính xác hơn.

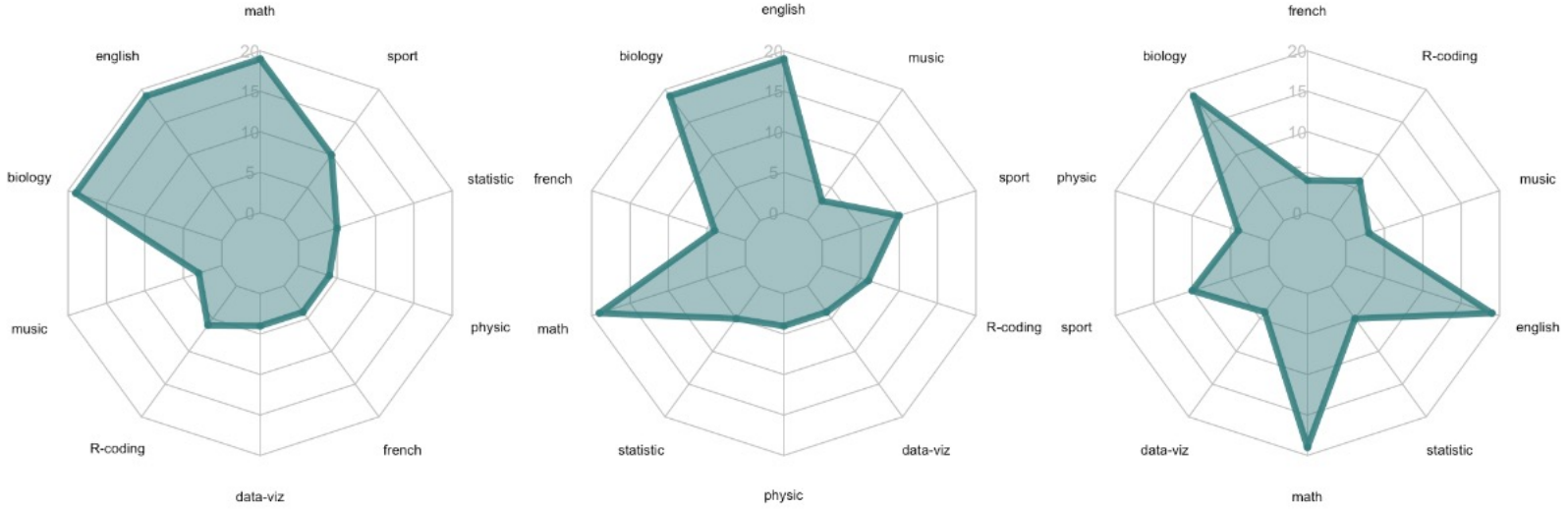




5. Nhược điểm của Spider chart

2) Hỗ trợ việc xếp hạng - Trong ví dụ trên, Hình lollipop được sắp xếp theo thứ tự. Thấy ngay được môn nào có điểm cao nhất và thứ hạng của từng môn. Điều này khó hơn khi dung spider chart.

3) Thứ tự danh mục có tác động rất lớn - Người đọc biểu đồ radar có thể sẽ tập trung vào hình dạng được quan sát. Điều này có thể gây hiểu nhầm vì hình dạng này phụ thuộc rất nhiều vào thứ tự của các danh mục xung quanh đồ thị. Xem các biểu đồ này được tạo bằng cách sử dụng cùng một dữ liệu nhưng thay đổi thứ tự danh mục:



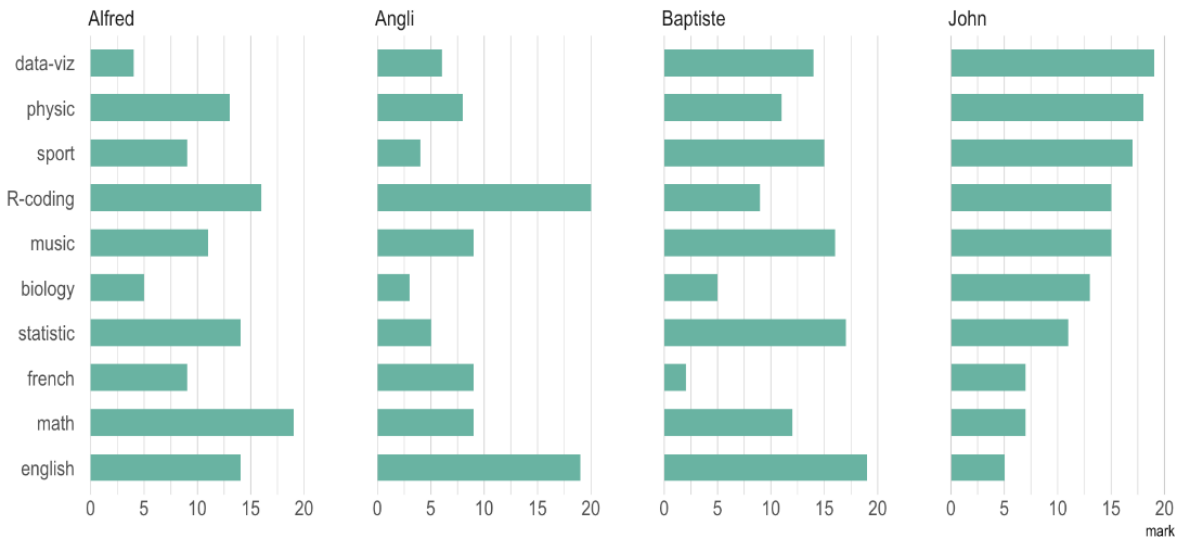
5. Nhược điểm của Spider chart

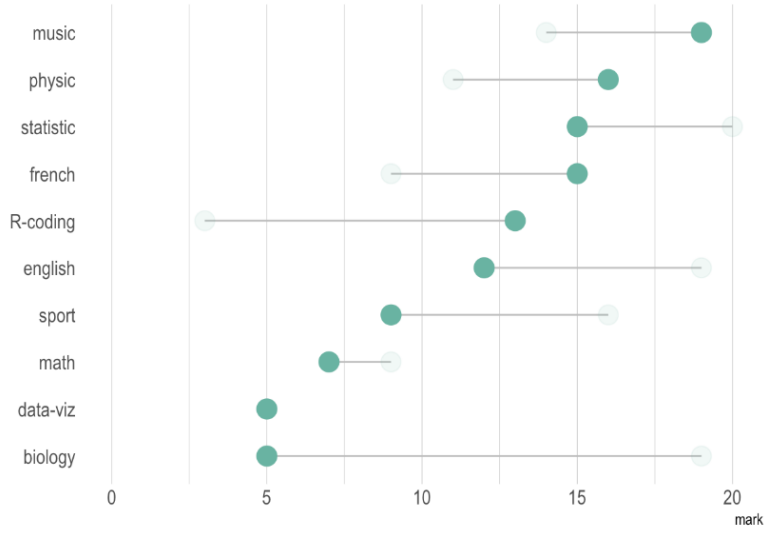
4) Về tỷ lệ - Biểu đồ radar hiển thị giá trị của một số biến định lượng, tất cả đều được biểu thị trên một trục. Trong ví dụ trước, cùng một thang đo và cùng một đơn vị được chia sẻ cho tất cả các biến (điểm nằm trong khoảng từ 0 đến 20). Cũng có thể hiển thị các biến hoàn toàn khác nhau. Trong trường hợp này, đừng quên hiển thị thang đo rõ ràng cho từng thang đo: nếu không thì người đọc sẽ hiểu lầm là cùng thang đo.



5. Nhược điểm của Spider chart

5) Vẽ quá nhiều - hiển thị nhiều hơn một vài biến sẽ dẫn đến hình ảnh không thể đọc được. Hãy tránh và thay vào đó hãy sử dụng số lượng biến nhỏ hơn 5. ➔ Cách giải quyết là dung biểu đồ khác ví dụ như bar chart hoặc lollipop chart.

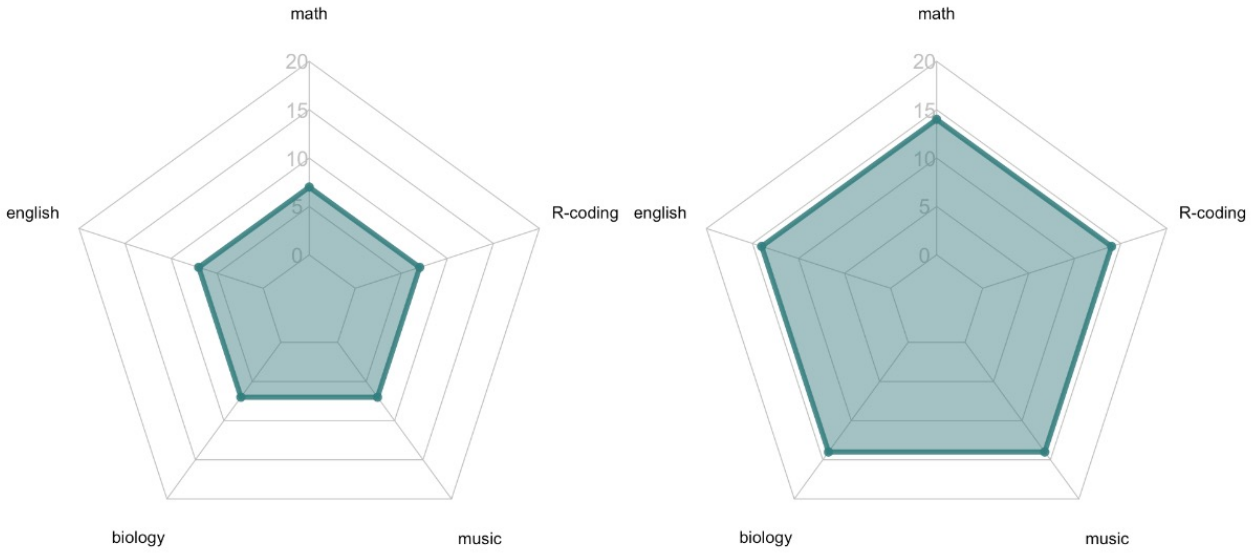




5. Nhược điểm của Spider chart

6) Đánh giá quá cao về sự khác biệt - diện tích của một hình trong biểu đồ radar cũng tăng theo phương trình bậc hai thay vì tuyến tính, điều này có thể khiến người xem nghĩ rằng những thay đổi nhỏ có ý nghĩa quan trọng hơn thực tế.

Vd dưới bên trên học sinh đạt 7 điểm cho tất cả các môn, bên phải đạt 14 điểm. Ta thấy diện tích bên phải lớn hơn nhiều so với 2 lần của bên trái.



6. Lưu ý khi dùng Spider chart

- **Số lượng biến**: Spider Chart không hiệu quả nếu có quá nhiều biến (hơn 10 biến), do khó đọc và hiểu.
- **Tỉ lệ hóa các trục**: Cần đảm bảo rằng các trục được tỉ lệ hóa đúng cách để tránh gây hiểu nhầm về mức độ khác biệt giữa các biến.
- **Hiển thị nhiều đối tượng**: Nếu so sánh quá nhiều đối tượng, cần sử dụng mã màu rõ ràng hoặc chia nhỏ biểu đồ để dễ phân tích.

7. Nhận xét từ Spider chart

- So sánh tổng quan và chi tiết giữa các đối tượng.
- Phát hiện điểm mạnh, điểm yếu của từng đối tượng.
- Đánh giá sự đồng đều hoặc không đồng đều trong hiệu suất.
- Nhận diện mối tương quan giữa các biến.
- Xác định yếu tố nào có sự biến động lớn nhất và ảnh hưởng đến sự khác biệt giữa các đối tượng.

7.1 So sánh tổng quan và chi tiết giữa các đối tượng

Dễ thấy được đối tượng nào vượt trội hơn về tổng thể, hoặc ở các biến cụ thể. Ví dụ, nếu một đối tượng có diện tích phủ rộng hơn so với các đối tượng khác, điều đó có nghĩa là nó có hiệu suất cao hơn trên hầu hết các biến.



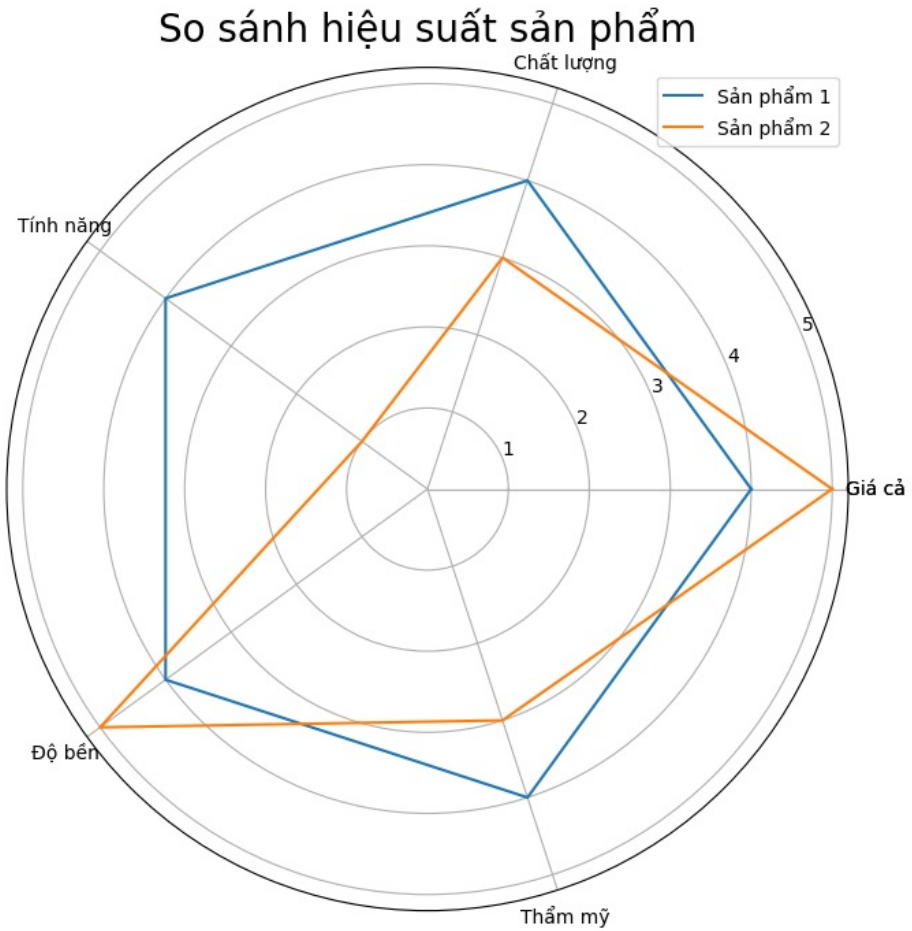
7.2 Phân tích điểm mạnh và yếu

- Nhận xét: Các trục biểu diễn các biến số cho thấy những khu vực mà đối tượng thể hiện tốt hoặc kém.
- Insight: Từ đó, ta có thể tập trung cải thiện các điểm yếu hoặc khai thác tối đa các điểm mạnh của đối tượng.
- Ví dụ, trong đánh giá kỹ năng của nhân viên, một nhân viên có thể rất giỏi về giao tiếp nhưng yếu về quản lý thời gian, điều này được phản ánh qua việc điểm số trên trục "Quản lý thời gian" gần tâm hơn so với các trục khác.



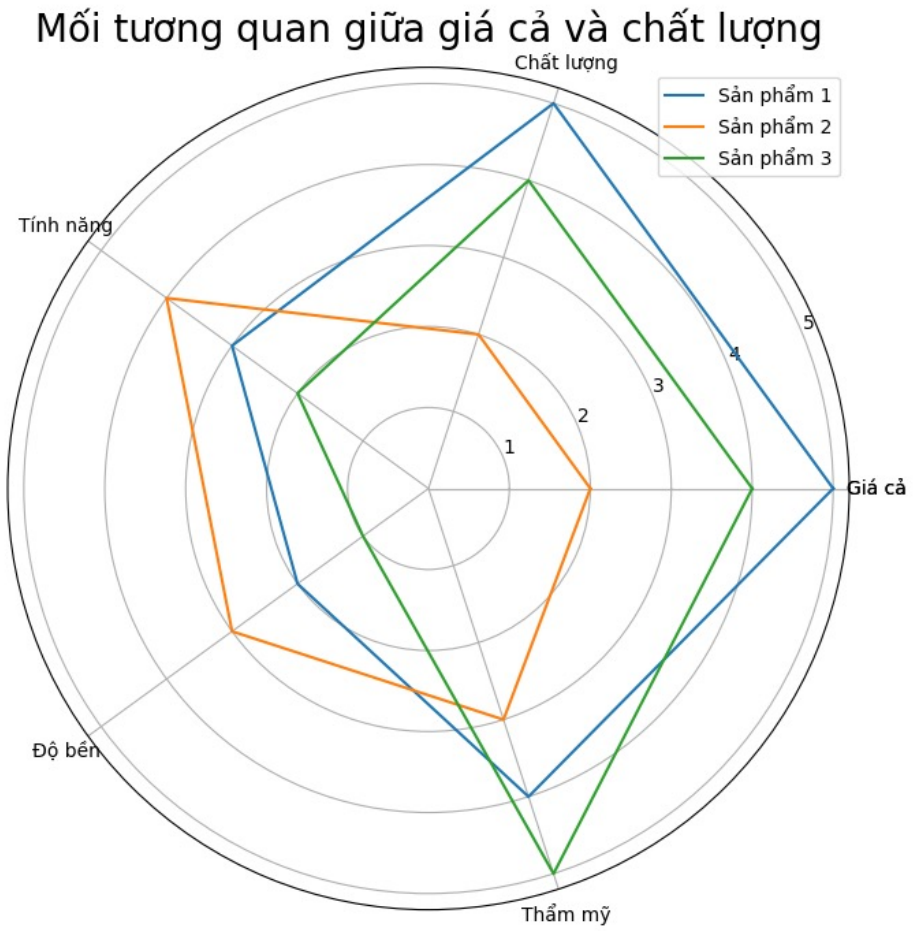
7.3 Phân tích sự cân bằng hoặc mất cân đối

- **Nhận xét**: Hình dạng của biểu đồ cho thấy mức độ cân bằng giữa các tiêu chí. Một biểu đồ cân đối cho thấy đối tượng có hiệu suất đồng đều trên các biến số.
- **Insight**: Nếu biểu đồ có những phần nhô cao hoặc thấp, điều này chỉ ra sự mất cân đối, giúp ta hiểu rõ hơn về cách phân bổ tài nguyên hoặc nỗ lực.



7.4 Nhận diện các mối tương quan giữa các biến

- **Nhận xét**: Dạng hình của biểu đồ có thể phản ánh mối tương quan giữa các biến số. Nếu một số trục có xu hướng tăng hoặc giảm cùng nhau, có thể tồn tại mối liên hệ giữa chúng.
- **Insight**: Điều này giúp ta hiểu rõ hơn về cấu trúc dữ liệu và các yếu tố ảnh hưởng lẫn nhau.



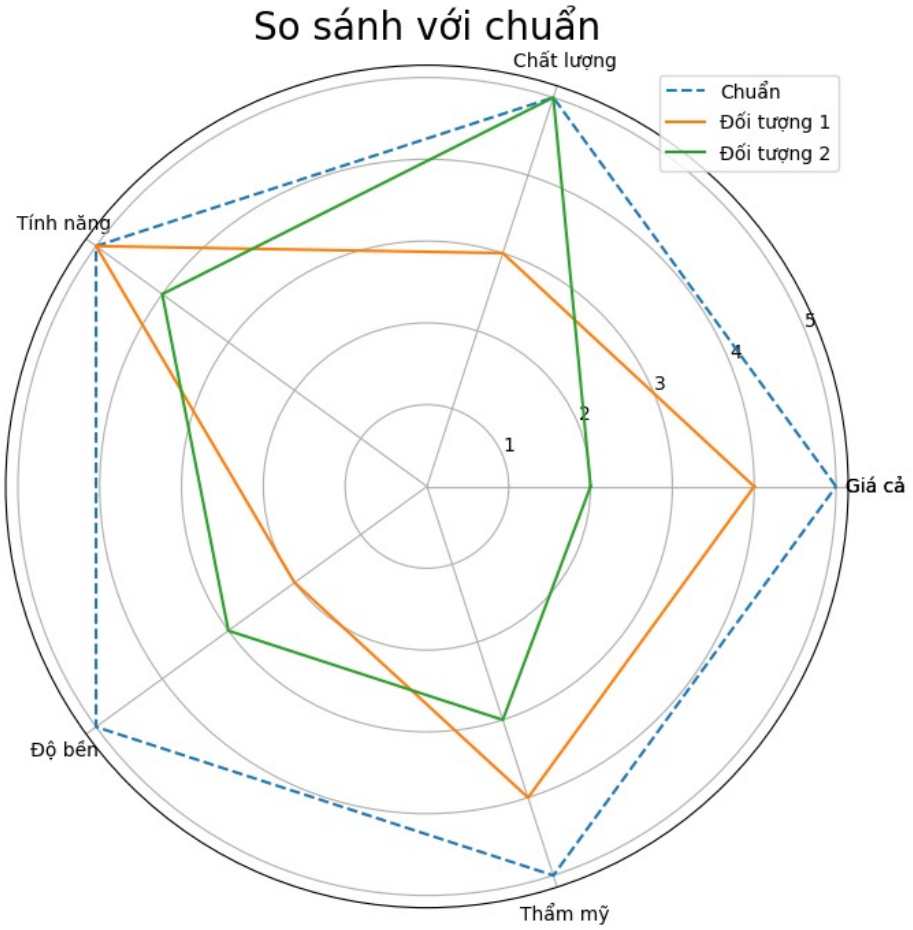
7.5 Phát hiện sự vượt trội hoặc tụt hậu

- **Nhận xét**: Các đối tượng có các trục đạt giá trị cao hơn hoặc thấp hơn đồng đều có thể được xem là vượt trội hoặc tụt hậu trong tổng thể.
- **Insight**: Đây là cơ sở để đưa ra quyết định chiến lược, như chọn lựa đối tác, cải thiện sản phẩm, hoặc đánh giá hiệu suất cá nhân.



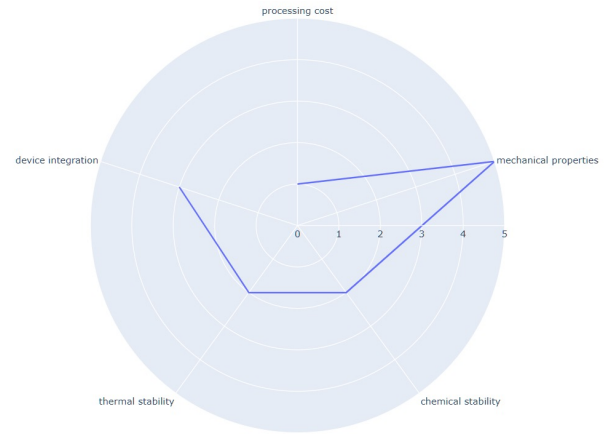
7.6 Phân tích mức độ phù hợp với mục tiêu

- **Nhận xét**: So sánh biểu đồ Spider của đối tượng với một biểu đồ chuẩn hoặc mục tiêu đề ra.
- **Insight**: Giúp ta xác định xem đối tượng có đáp ứng được các yêu cầu hoặc tiêu chuẩn đã đặt ra hay không, từ đó điều chỉnh chiến lược phù hợp.



8. Vẽ 1 biến với plotly

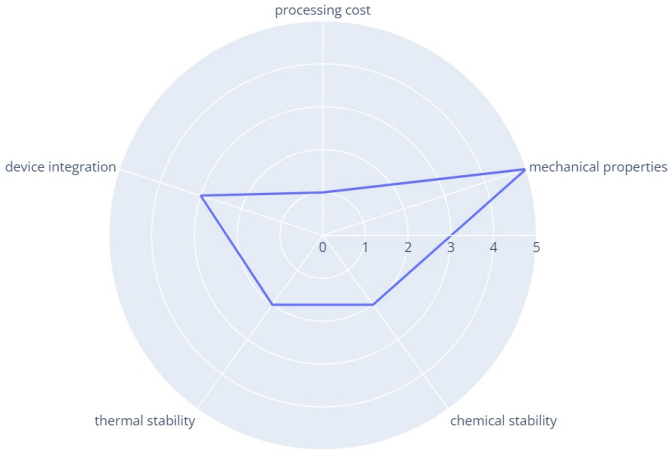
- Dạng spider đơn giản nhất là vẽ 1 biến, ta sẽ sử dụng thư viện plotly.




```python
import plotly.express as px
import pandas as pd

data = {
    "r": [1, 5, 2, 2, 3],
    "theta": ['processing cost', 'mechanical properties',
              'chemical stability','thermal stability',
              'device integration']
}
df = pd.DataFrame(data)

fig = px.line_polar(df, r='r', theta='theta', line_close=True)
fig.show()
```



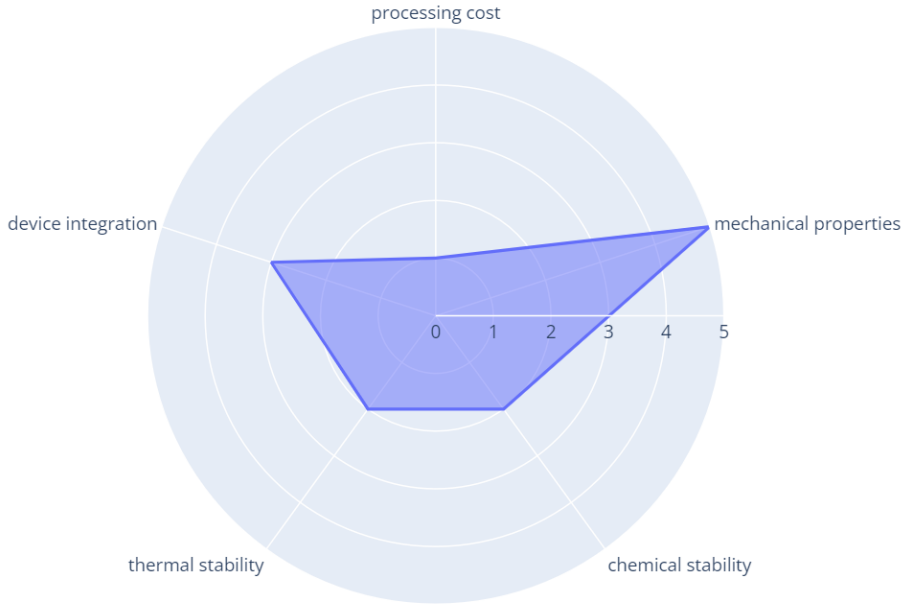
8. Vẽ 1 biến với plotly

- Ta có thể đổ màu cho hình

```python
import plotly.express as px
import pandas as pd

data = {
    "r": [1, 5, 2, 2, 3],
    "theta": ['processing cost', 'mechanical properties',
              'chemical stability','thermal stability',
              'device integration']
}
df = pd.DataFrame(data)

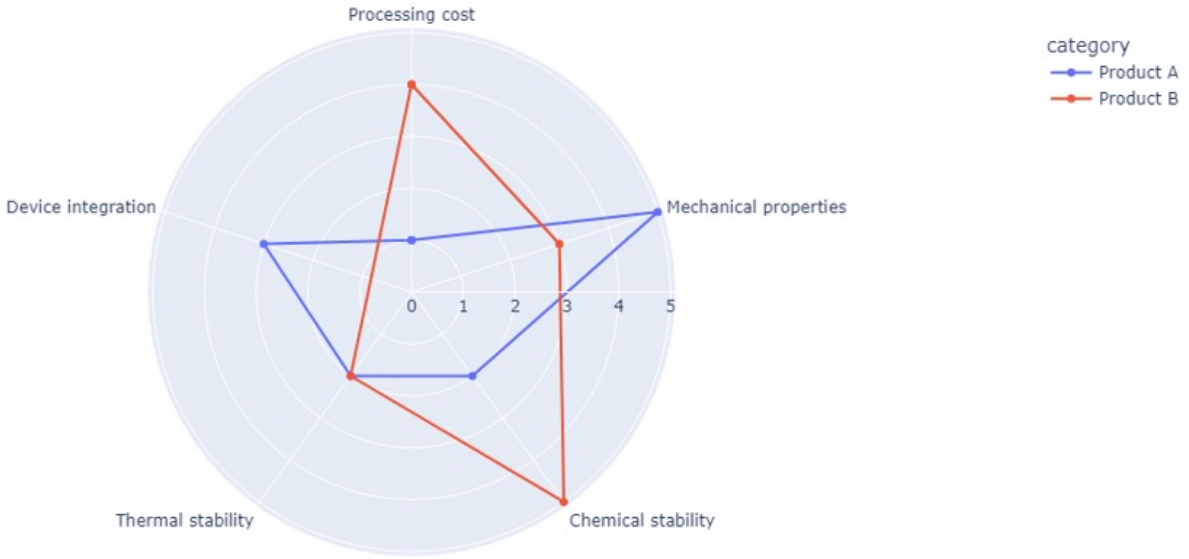
fig = px.line_polar(df, r='r', theta='theta', line_close=True)
fig.update_traces(fill='toself')
fig.show()
```



9. Vẽ nhiều biến

```python
import plotly.express as px
import pandas as pd

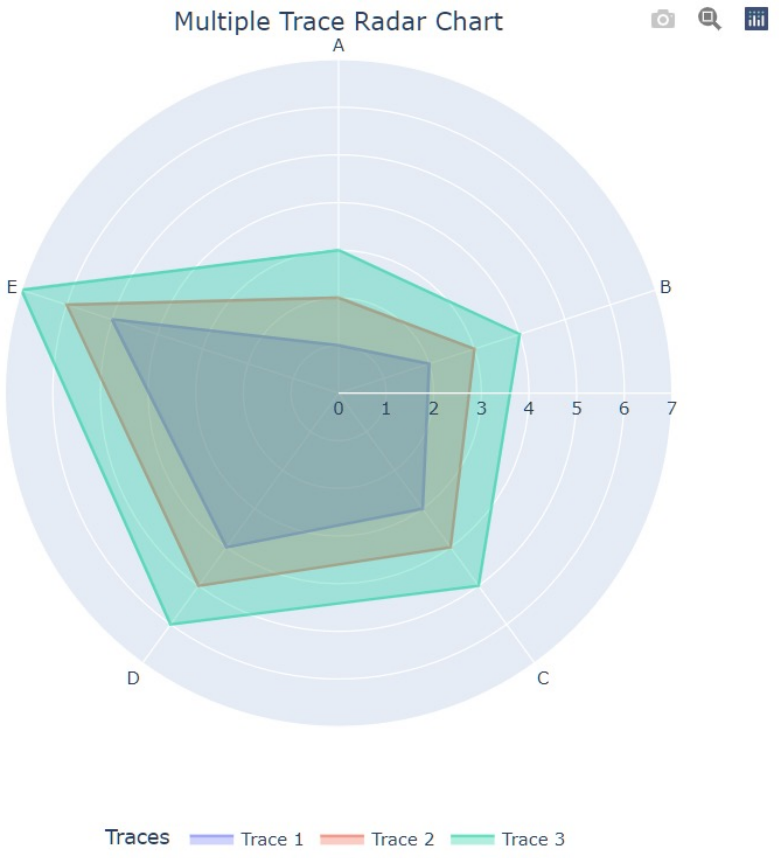
df = pd.DataFrame(dict(
    r=[1, 5, 2, 2, 3, 4, 3, 5, 2],
    theta=['Processing cost', 'Mechanical properties','Chemical stability', 'Thermal stability', 'Device integration',
           'Processing cost', 'Mechanical properties','Chemical stability', 'Thermal stability'],
    category=['Product A', 'Product A', 'Product A', 'Product A', 'Product A', 'Product B', 'Product B', 'Product B', 'Product B']
))
fig = px.line_polar(df, r='r', theta='theta', line_close=True, color='category', markers=True)
fig.show()
```



9. Vẽ nhiều biến

- Căn chỉnh layout and legend, figure size.

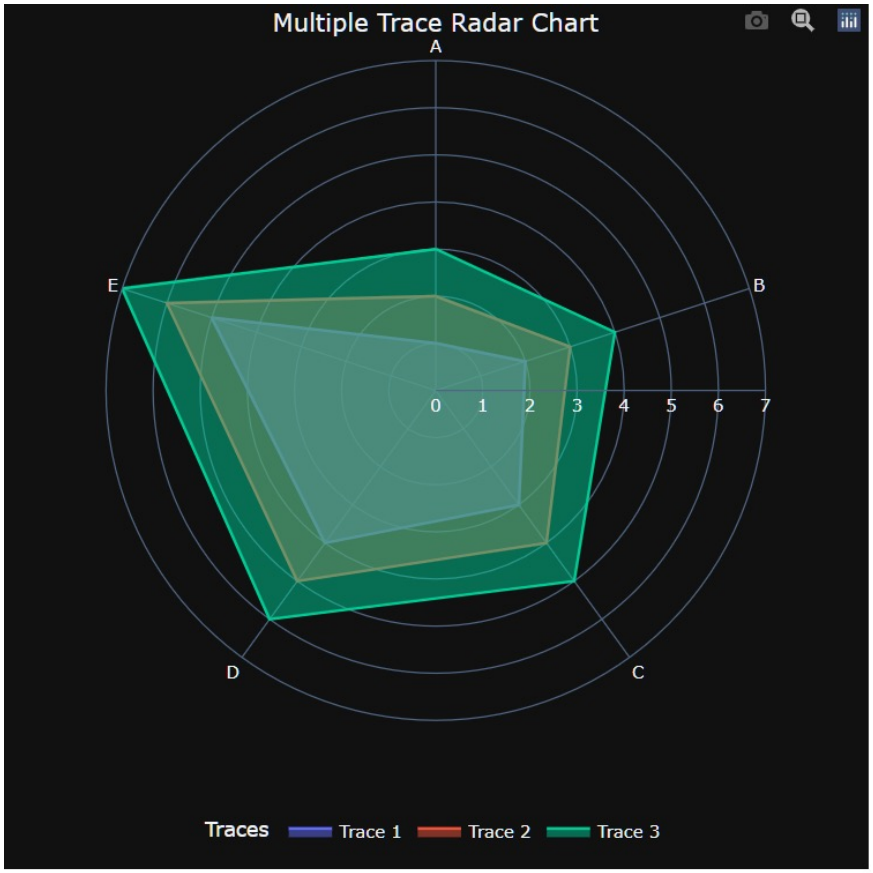
```python
import pandas as pd
import plotly.express as px
data = {
    'Category': ['A', 'B', 'C', 'D', 'E'] * 3,
    'Value': [1, 2, 3, 4, 5, 2, 3, 4, 5, 6, 3, 4, 5, 6, 7],
    'Trace': ['Trace 1'] * 5 + ['Trace 2'] * 5 + ['Trace 3'] * 5
}
df = pd.DataFrame(data)
# Create the radar chart
fig = px.line_polar(df, r='Value', theta='Category', color='Trace', line_close=True)
# Customize the layout of the radar chart
fig.update_traces(fill='toself', opacity=0.6)
fig.update_layout(
    title='Multiple Trace Radar Chart',
    title_x=0.5,  # Center the title
    legend=dict(
        title='Traces',
        orientation='h',
        xanchor='center',
        x=0.5,
        yanchor='bottom',
        y=-0.2
    ),
    margin=dict(l=40, r=40, b=40, t=40),  # Set the margins
    width=600,  # Set the width of the figure
    height=600  # Set the height of the figure
)
fig.show()
```



9. Vẽ nhiều biến

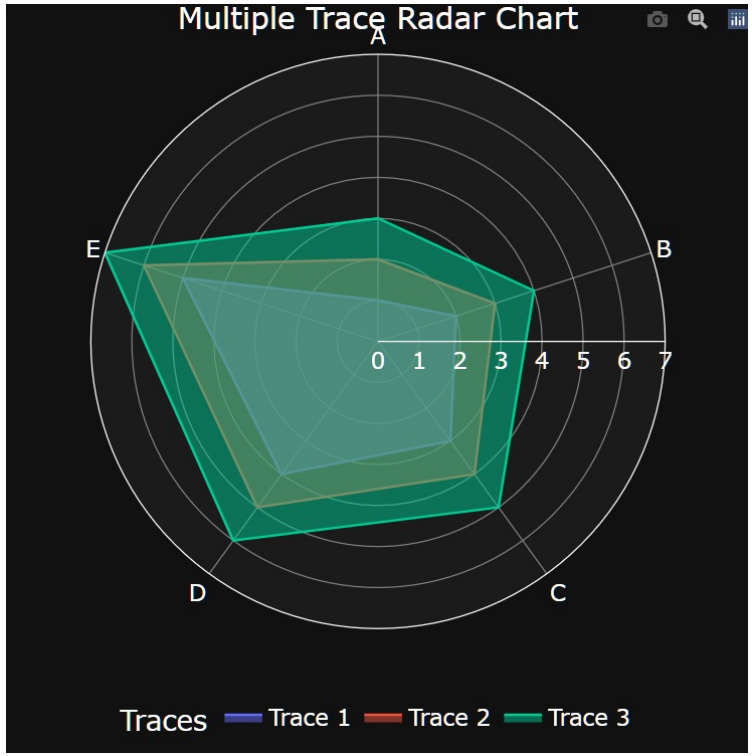
➢ Chọn cách style khác nhau

```python
import pandas as pd
import plotly.express as px
data = {
    'Category': ['A', 'B', 'C', 'D', 'E'] * 3,
    'Value': [1, 2, 3, 4, 5, 2, 3, 4, 5, 6, 3, 4, 5, 6, 7],
    'Trace': ['Trace 1'] * 5 + ['Trace 2'] * 5 + ['Trace 3'] * 5
}
df = pd.DataFrame(data)
# Create the radar chart
fig = px.line_polar(df, r='Value', theta='Category', color='Trace', line_close=True)
# Customize the layout of the radar chart
fig.update_traces(fill='toself')
fig.update_layout(
    title='Multiple Trace Radar Chart',
    title_x=0.5,  # Center the title
    legend=dict(
        title='Traces',
        orientation='h',
        xanchor='center',
        x=0.5,
        yanchor='bottom',
        y=-0.2
    ),
    margin=dict(l=40, r=40, b=40, t=40),  # Set the margins
    width=600,  # Set the width of the figure
    height=600,  # Set the height of the figure
    template='plotly_dark'  # Set the theme: ggplot2, seaborn, simple_white, plotly
)
fig.show()
```



9. Vẽ nhiều biến

- Chỉnh sửa font, size, màu sắc.




```python
import pandas as pd
import plotly.express as px
data = {
    'Category': ['A', 'B', 'C', 'D', 'E'] * 3,
    'Value': [1, 2, 3, 4, 5, 2, 3, 4, 5, 6, 3, 4, 5, 6, 7],
    'Trace': ['Trace 1'] * 5 + ['Trace 2'] * 5 + ['Trace 3'] * 5
}
df = pd.DataFrame(data)
# Create the radar chart
fig = px.line_polar(df, r='Value', theta='Category', color='Trace', line_close=True)
# Customize the layout of the radar chart
fig.update_traces(fill='toself')
fig.update_layout(
    title='Multiple Trace Radar Chart',
    title_x=0.5,  # Center the title
    legend=dict(
        title='Traces',
        orientation='h',
        xanchor='center',
        x=0.5,
        yanchor='bottom',
        y=-0.2
    ),
    margin=dict(l=40, r=40, b=40, t=40),  # Set the margins
    width=600,  # Set the width of the figure
    height=600,  # Set the height of the figure
    template='plotly_dark'  # Set the theme: ggplot2, seaborn, simple_white, plotly
)
## Set up font, size, color
fig.update_traces(marker=dict(size=12, line=dict(width=2, color='DarkSlateGrey')))
fig.update_layout(
    title=dict(font=dict(size=24, color='white')),
    polar=dict(
        bgcolor='rgba(30,30,30,0.8)',  # Set background color of polar plot
        angularaxis=dict(linecolor='white', gridcolor='gray'),
        radialaxis=dict(linecolor='white', gridcolor='gray')
    ),
    font=dict(size=18, color='white')  # Set font for all text
)
fig.show()
```

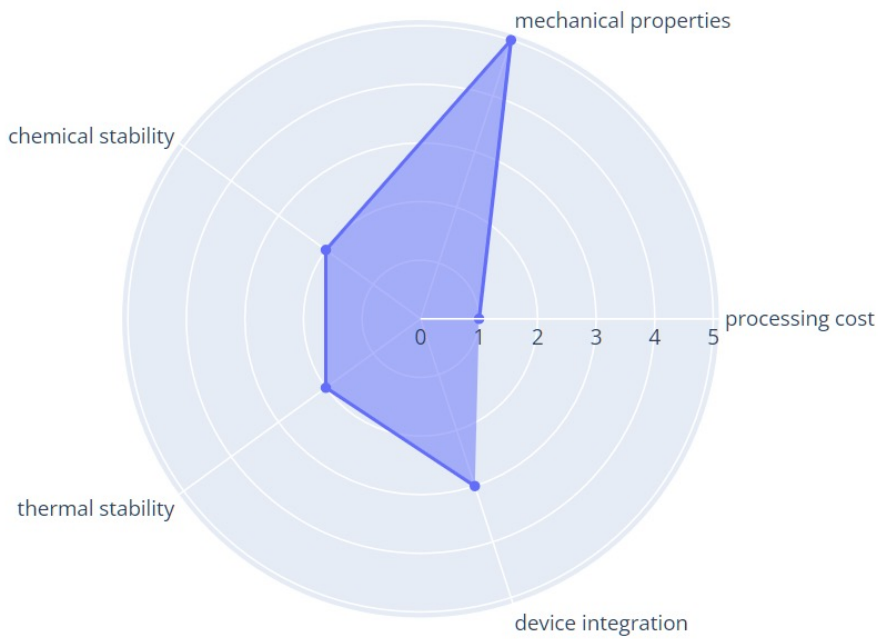
10. Vẽ 1 biến với Scatterpolar

➢ Có một vấn đề với Scatterpolar đó là không có tham số line_close. Để đóng được hình ta phải thêm giá trị đầu tiên vào cuối dữ liệu. Lúc đó điểm cuối sẽ trùng điểm đầu và hình sẽ được đóng lại.

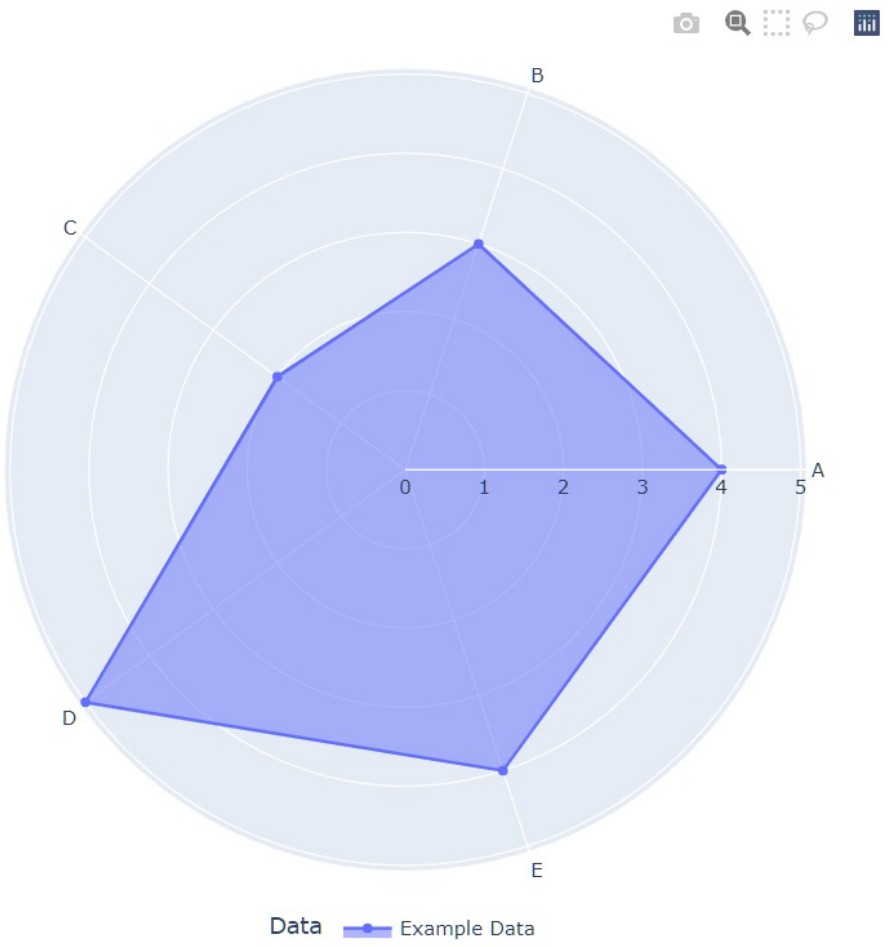
```
import plotly.graph_objects as go

fig = go.Figure(
    data=go.Scatterpolar(
        r=[1, 5, 2, 2, 3],
        theta=['processing cost', 'mechanical properties',
               'chemical stability', 'thermal stability',
               'device integration'],
        fill='toself'
    )
)
fig.update_layout(
    showlegend=True
)

fig.show()
```



10. Vẽ 1 biến với Scatterpolar




```python
import plotly.graph_objects as go

# Example data
categories = ['A', 'B', 'C', 'D', 'E']
values = [4, 3, 2, 5, 4]

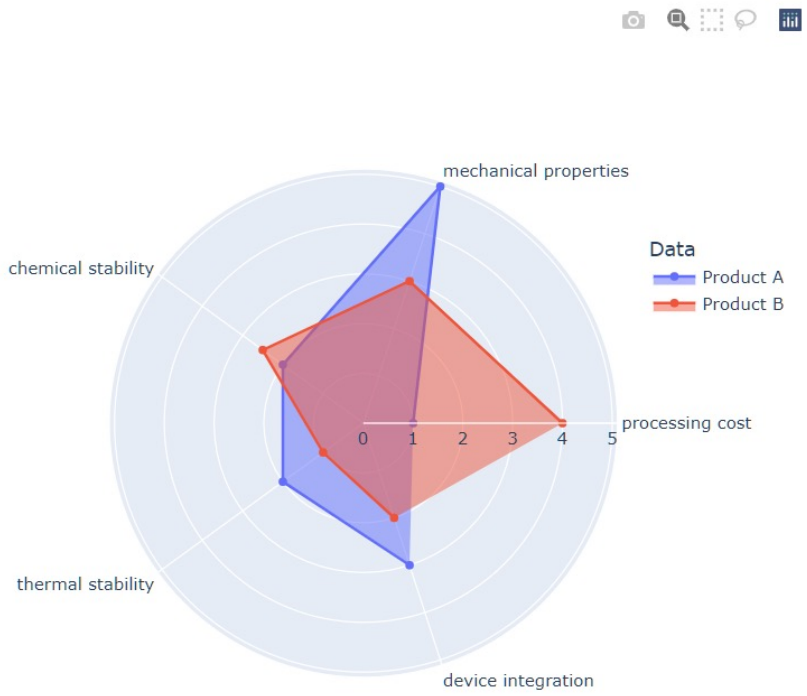
# Close the loop by appending the first value at the end
values.append(values[0])
categories.append(categories[0])

# Create the radar chart
fig = go.Figure()

fig.add_trace(go.Scatterpolar(
    r=values,
    theta=categories,
    fill='toself',
    name='Example Data'
))

# Update layout for a better visual
fig.update_layout(
    showlegend=True,
    legend=dict(
        title='Data',
        orientation='h',
        xanchor='center',
        x=0.5,  # Adjust x position to move the legend to the center
        yanchor='bottom',
        y=-0.1  # Adjust y position to center vertically
    ),
    width=600,  # Adjust width as needed
    height=600,  # Adjust height as needed
    margin=dict(l=40, r=40, t=40, b=40)  # Tight margins
)
fig.show()
```

10. Vẽ nhiều biến




```python
import plotly.graph_objects as go

categories = ['processing cost', 'mechanical properties',
              'chemical stability', 'thermal stability',
              'device integration']

fig = go.Figure()
fig.add_trace(go.Scatterpolar(
    r=[1, 5, 2, 2, 3],
    theta=categories,
    fill='toself',
    name='Product A'
))
fig.add_trace(go.Scatterpolar(
    r=[4, 3, 2.5, 1, 2],
    theta=categories,
    fill='toself',
    name='Product B'
))

fig.update_layout(
    showlegend=True,
    legend=dict(
        title='Data',
        orientation='v',        # vertically
        xanchor='center',
        x=1.2,  # Adjust x position
        yanchor='middle',
        y=0.7  # Adjust y position to center
    ),
    width=600,  # Adjust width as needed
    height=600,  # Adjust height as needed
    margin=dict(l=100, r=40, t=40, b=40)  # Tight margins
)
fig.show()
```

10. Vẽ nhiều biến trên nhiều hình

```python
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

data = {
    'Category': ['A', 'B', 'C', 'D', 'E'],
    'Product A': [1, 2, 3, 4, 5],
    'Product B': [2, 3, 4, 5, 6],
    'Product C': [3, 4, 5, 6, 7]
}
df = pd.DataFrame(data)

# Initialize the subplot figure
fig = make_subplots(
    rows=1, cols=3,
    specs=[[{'type': 'polar'}, {'type': 'polar'}, {'type': 'polar'}]],
    subplot_titles=('Product A', 'Product B', 'Product C')
)

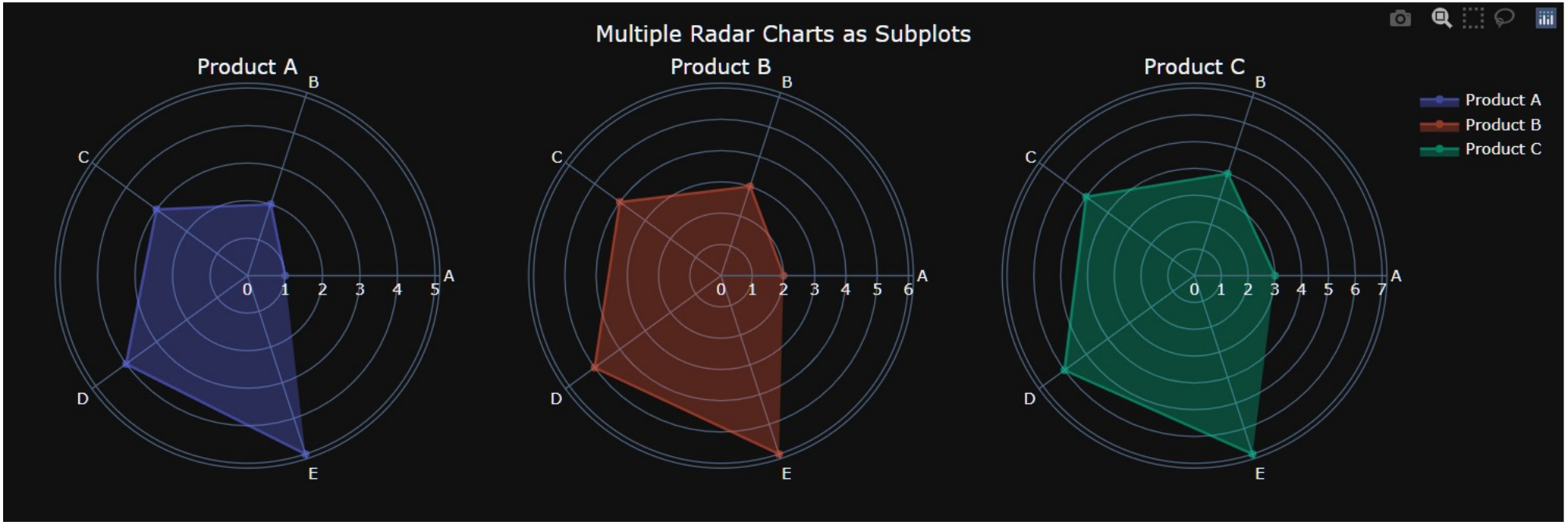
# Add traces for each subplot
fig.add_trace(go.Scatterpolar(
    r=df['Product A'],
    theta=df['Category'],
    fill='toself',
    name='Product A'
), row=1, col=1)

fig.add_trace(go.Scatterpolar(
    r=df['Product B'],
    theta=df['Category'],
    fill='toself',
    name='Product B'
), row=1, col=2)

fig.add_trace(go.Scatterpolar(
    r=df['Product C'],
    theta=df['Category'],
    fill='toself',
    name='Product C'
), row=1, col=3)

# Customize the layout of the radar charts
fig.update_traces(opacity=0.6)
fig.update_layout(
    title='Multiple Radar Charts as Subplots',
    title_x=0.5,  # Center the title
    width=1200,  # Set the width of the figure
    height=400,  # Set the height of the figure
    margin=dict(l=40, r=40, b=40, t=60),  # Set the margins
    template='plotly_dark'  # Set the theme
)
# Show the radar charts as subplots
fig.show()
```

10. Vẽ nhiều biến trên nhiều hình



10. Vẽ nhiều biến trên nhiều hình

```python
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Example data
data = {
    'Category': ['A', 'B', 'C', 'D', 'E'],
    'Product A': [1, 2, 3, 4, 5],
    'Product B': [2, 3, 4, 5, 6],
    'Product C': [3, 4, 5, 6, 7],
    'Product D': [4, 5, 6, 7, 8]
}

df = pd.DataFrame(data)

# Initialize the subplot figure with reduced spacing
fig = make_subplots(
    rows=2, cols=2,
    specs=[[{'type': 'polar'}, {'type': 'polar'}],
           [{'type': 'polar'}, {'type': 'polar'}]],
    subplot_titles=('Product A', 'Product B', 'Product C', 'Product D'),
    vertical_spacing=0.1,  # Reduce vertical spacing
    horizontal_spacing=0.1  # Reduce horizontal spacing
)

# Add traces for each subplot
fig.add_trace(go.Scatterpolar(
    r=df['Product A'],
    theta=df['Category'],
    fill='toself',
    name='Product A'
), row=1, col=1)

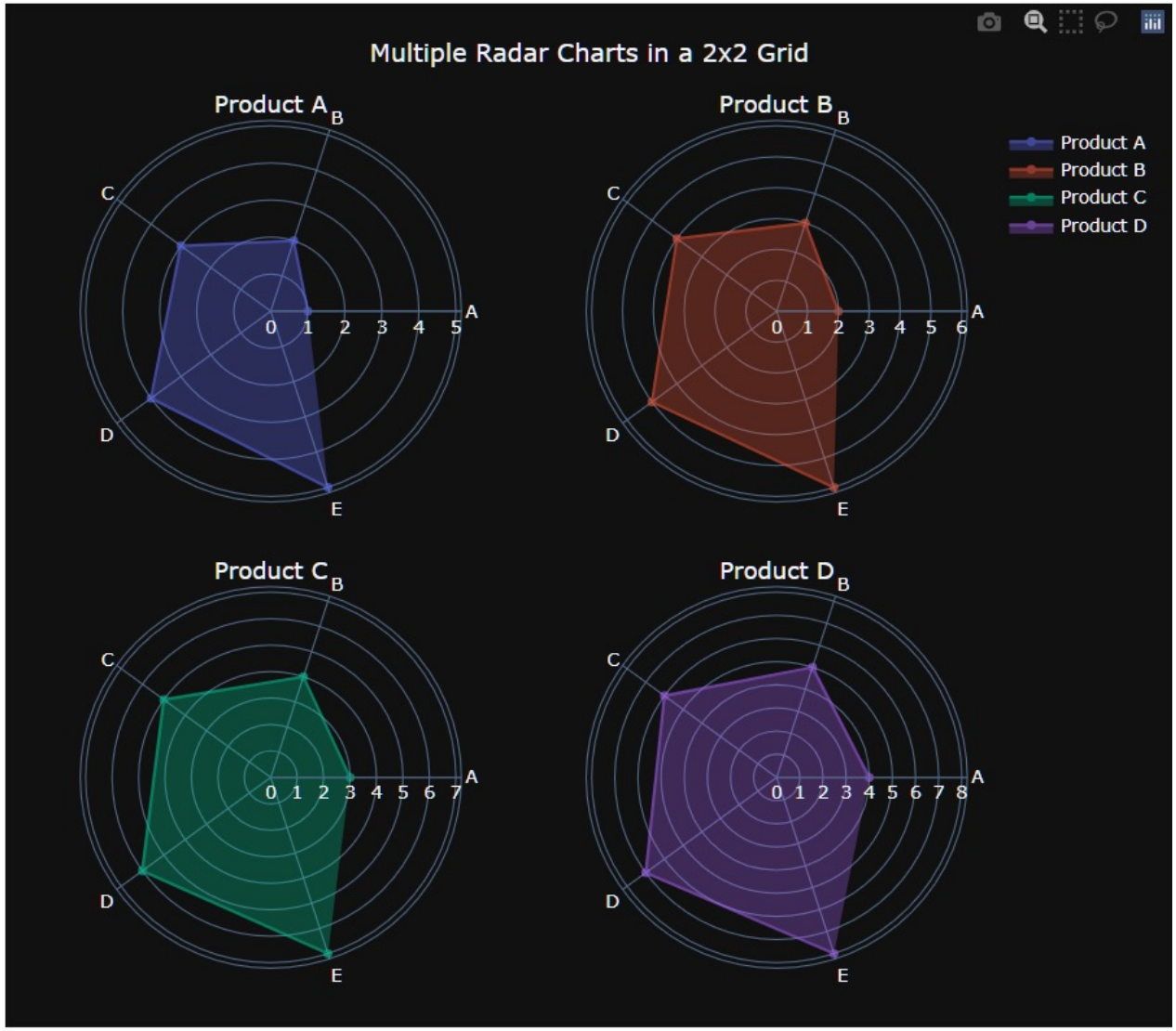
fig.add_trace(go.Scatterpolar(
    r=df['Product B'],
    theta=df['Category'],
    fill='toself',
    name='Product B'
), row=1, col=2)

fig.add_trace(go.Scatterpolar(
    r=df['Product C'],
    theta=df['Category'],
    fill='toself',
    name='Product C'
), row=2, col=1)

fig.add_trace(go.Scatterpolar(
    r=df['Product D'],
    theta=df['Category'],
    fill='toself',
    name='Product D'
), row=2, col=2)

# Customize the layout of the radar charts
fig.update_traces(opacity=0.6)
fig.update_layout(
    title='Multiple Radar Charts in a 2x2 Grid',
    title_x=0.5,  # Center the title
    width=800,  # Set the width of the figure
    height=700,  # Set the height of the figure
    margin=dict(l=40, r=40, b=40, t=80),  # Set the margins
    template='plotly_dark'  # Set the theme
)
# Show the radar charts as subplots
fig.show()
```

10. Vẽ nhiều biến trên nhiều hình



11. Bài tập

Sử dụng tập dữ liệu “Salaries.csv” (Nằm trong folder data https://github.com/thieu1995/csv-files). Vẽ hình Spider và đánh giá nội dung của hình vẽ.

+ Các categories có thể là cột discipline.

+ Các thể loại cần so sánh có thể là cột rank (Prof, AssocProf, AsstProf)

+ Hoặc có thể so sánh giữa cột sex (Male và Female)